Exercise - 8 Solutions

Adversarial Attacks

Exercise 8.1: What Are Adversarial Examples? Explain what an adversarial example is. How do they differ from Out-of-Distribution examples?

→
An adversarial example is an input that has been intentionally changed in a small way so that the model gives an incorrect output.
The change may be almost invisible to a human, but it can be enough to move the input across the model's decision boundary.

It may still look like the same object to a human, but after adding a small perturbation, the model may classify it as a completely different class.

Difference btw Adversarial and OOD →

Aspect	Adversarial example	Out-of-Distribution example
Meaning	A normal-looking input intentionally modified to fool the model	An input that naturally lies outside the training/valid operating distribution
Cause	Usually created by an attacker or optimization process	Caused by domain change, unseen condition – distribution shift
Example	Sunny road image with tiny perturbation that hides a pedestrian from the model	Fog image, night image, or unseen town layout
Human perception	Often looks almost the same as the original image	Often visibly different from training data
Relation to training distribution	Usually close to an in-distribution sample	Farther away from the training distribution
Main problem	Model is brittle to small, targeted changes	Model is unreliable outside its validated ODD

Exercise 8.2: Attack Formulation

A basic gradient-based attack updates the input as:

$$x_{i+1} = x_i + \alpha \nabla_x \mathcal{L}(y, f(x_i))$$

1. What does each term represent?

→ This is a basic gradient-based adversarial attack. The attacker updates the input in the direction that increases the model loss.
Adversarial attacks as optimizing the input, not the model weights, for example to maximize loss.

Term	Meaning
xi	Current input at iteration (i). This can be the original image or the already slightly modified image.
$xi + 1$	Updated adversarial input after one attack step.
α	Step size or learning rate. It controls how large the update is in each step.
y	True label of the input, for example “pedestrian present”
$f(xi)$	Model prediction/output for the current input (x_i).
$L(y, f(xi))$	Loss function comparing the true label (y) with the model output (f(x_i)). Higher loss means the model is performing worse.
$\nabla xL(y, f(xi))$	Gradient of the loss with respect to the input image. Tells which direction to change the pixels to increase the loss.

2. What is the difference between a targeted and an untargeted attack? How would the formula change for a targeted attack?

→
Untargeted attack - An untargeted attack only wants the model to make a wrong prediction. It does not care which wrong class the model chooses.

For an untargeted attack, we maximize the loss for the true label:

$$x_{i+1} = x_i + \alpha \nabla_x L(y, f(x_i))$$

The attack moves the input in the direction that makes the model worse for the correct label y.

Targeted attack - A targeted attack wants the model to predict a specific wrong target class.

For a targeted attack, we define a target class: $y\ target$

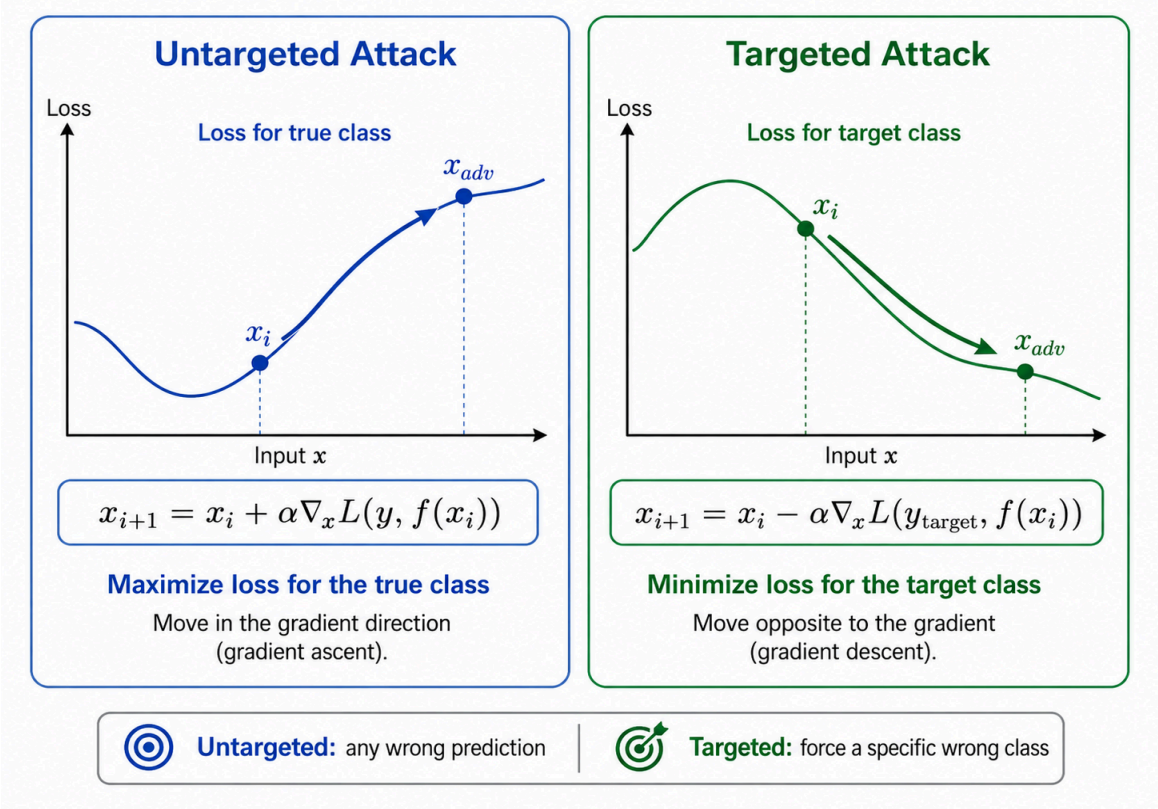
Then we minimize the loss for the target class, so that the model becomes more confident in the attacker-chosen class

$$x_{i+1} = x_i - \alpha \nabla_x L(y_{target}, f(x_i))$$

The sign changes from plus to minus because we are now doing gradient descent on the target loss.

Difference →

Attack type	Goal	Formula idea
Untargeted	Make the model wrong	Maximize loss for true class
Targeted	Make the model predict a specific wrong class	Minimize loss for target class



3. Why does this basic update rule not respect a perturbation budget $\| x0 - xt \| \leq \varepsilon$? How can it be modified to do so?

→

The basic update rule does not respect the perturbation budget because it only follows the gradient direction that increases the loss. It does not check whether the adversarial input is still close to the original input x0.

Therefore, after several iterations, the distance between the original image and the adversarial image may become larger than the allowed budget ϵ.

To fix this, we can add a projection step after every update. After taking a gradient step, the new input is projected back into the allowed ϵ-ball around the original image. The modified rule is:

$$x_{i+1} = \Pi_{B_\epsilon(x_0)} \left(x_i + \alpha \nabla_x L(y, f(x_i)) \right)$$

For an L∞ constraint, this projection can be implemented by clipping every pixel so that it stays between x0−ϵ and x0+ϵ. This keeps the adversarial image close to the clean image while still trying to fool the model.

Exercise 8.3: Defenses

Explain the idea of adversarial training. What trade-off does it introduce?

→ Adversarial training is a defense method where the model is trained not only on clean images, but also on adversarially perturbed images.

The idea is:

1. Take a normal training image.
2. Generate an adversarial version of it, for example using FGSM or PGD.
3. Train the model to classify both the clean and adversarial image correctly.

It is a min-max problem: the inner step finds the worst-case adversarial input inside an ϵ-ball, and the outer step updates the model weights to be robust against that worst-case input.

→ The main trade-off is:

Benefit	Cost
Better robustness against adversarial examples	Lower accuracy on normal clean data
More stable decision boundaries	More expensive training
Better resistance within the chosen perturbation budget	May not generalize to different attacks or larger perturbations

Adversarial training can **decrease performance on normal data** and may only be robust within the chosen perturbation region B(x,ϵ)

Exercise 8.4: Generating Adversarial Examples

Load your trained CARLA model and generate adversarial examples using the Fast

Gradient Sign Method (FGSM):

$x_{adv} = x + \varepsilon \cdot \text{sign}(\nabla_x L(y, f(x))$

1. Implement FGSM for each of your three binary classifiers.

→ (ref ipynb for implementation)

BCEWithLogitsLoss is used because each model is a binary classifier. The loss measures how wrong the model prediction is compared to the true label. Then loss.backward() computes the gradient of this loss with respect to the input image pixels. This gradient tells us how each pixel should be changed to increase the loss.
FGSM then takes the sign of this gradient and adds a small perturbation of size ϵ to the original image. Therefore, the adversarial image is created by moving each pixel slightly in the direction that makes the model more likely to make a wrong prediction.

2. Generate adversarial examples for $\varepsilon \in \{0.01, 0.05, 0.1\}$.

→ (ref ipynb for implementation)

3. Display a clean image and its adversarial counterpart side-by-side. At what ε do the perturbations become visible to a human?

→ (ref ipynb for implementation)

Epsilon	Observation
0.01	Perturbation is almost invisible
0.05	Small noise starts becoming visible when comparing clean and adversarial images side-by-side
0.1	Perturbation is clearly visible as noise or color distortion

Exercise 8.5: Measuring Robustness

Evaluate each of your three models on the adversarial test set for each ε. (You may also use a subset of 100 randomly sampled images for evaluation.) Report the per-model recall drop compared to clean inputs.

→ (ref ipynb for implementation)

	Model	Epsilon	Clean Recall	Adversarial Recall	Recall Drop
0	Pedestrian Detector	0.01	0.3385	0.0000	0.3385
1	Pedestrian Detector	0.05	0.3385	0.0142	0.3244
2	Pedestrian Detector	0.10	0.3385	0.1686	0.1700
3	Traffic Light Detector	0.01	0.9717	0.0008	0.9710
4	Traffic Light Detector	0.05	0.9717	0.0008	0.9710
5	Traffic Light Detector	0.10	0.9717	0.0008	0.9710
6	Vehicle Detector	0.01	0.8893	0.2141	0.6752
7	Vehicle Detector	0.05	0.8893	0.3178	0.5715
8	Vehicle Detector	0.10	0.8893	0.1948	0.6944

Exercise 8.6: Extending the Safety Analysis for Adversarial Robustness

You have now measured how much adversarial perturbations degrade your models' recall. Revisit your System-Theoretic Process Analysis from Sheet 2 and extend it to cover this threat. Many of you will not yet have an adversarial-attack entry in your analysis; the goal of this exercise is to add one and trace it through.

1. Hazard. Check the hazards from Exercise 2.4. Is a degraded perception output caused by an adversarial input captured (e.g. as a cause of "vehicle fails to brake for a pedestrian")? If your hazard list does not reflect it, refine or add a hazard.

→

ID	Hazard	Loss(es)	Likelihood	Severity

H-1	Pedestrian in vehicle path while vehicle is moving without braking	L-1, L-2	Medium	High
H-2	Vehicle continues through intersection with undetected traffic signal	L-3, L-4	Medium	High
H-3	Vehicle fails to detect oncoming/crossing vehicle and does not yield	L-2, L-3	Medium	High
H-4	Autonomous mode remains active during ODD violation, e.g. night, rain	L-1, L-2, L-3	High	High
H-5	Vehicle brakes unnecessarily at high frequency due to false positives	L-2, L-3	Medium	Low
H-6	Vehicle continues operating on an undetected out-of-ODD input, such as fog, night, or unseen town layout, while perception models give confident but unreliable predictions	L-1, L-2, L-3, L-4, L-5	High	High
H-7	Vehicle continues operating while perception output is degraded by an undetected adversarial perturbation, causing false negatives for safety-critical objects such as pedestrians, traffic lights, or vehicles	L-1, L-2, L-3, L-4	Medium	High

→ H-7 is needed because adversarial perturbations can make a perception model fail even when the image still looks almost normal to a human. For example, a pedestrian image may be perturbed so that the pedestrian classifier outputs a false negative. This can directly lead to H-1, where the vehicle fails to brake for a pedestrian.

2. Unsafe control action. Review the UCAs from Exercise 2.6. Add a UCA for the case where the planner acts on perception output that is wrong because the input has been adversarially perturbed and this is not detected (e.g. “the planner continues at speed while the pedestrian classifier has been fooled by an adversarial attack and outputs a false negative”).

Link it to the hazard(s) above.

→

UCA ID	Controller	Control action	Unsafe control action	Linked hazard(s)

UCA-14	Planning module	Continue driving / maintain speed	The planner continues driving at normal speed while the perception output is wrong because the camera input has been adversarially perturbed and the attack is not detected.	**H-1, H-2, H-3, H-7**

→ This UCA is unsafe because the planner trusts the classifier output even though the input has been manipulated. For example, if the pedestrian detector is fooled and outputs “no pedestrian,” the planner may continue at speed instead of braking. This links directly to H-1 and H-7.

3. Safety constraints. From that UCA, derive at least one new safety constraint of each kind (cf. Exercise 2.7):

• a model-level constraint on adversarial robustness, referencing the ε budget and recall drop you measured; and

→

Constraint ID	Type	Safety constraint	Justification
SC-15	Model-level	The perception models shall maintain an acceptable recall under FGSM adversarial perturbations within the tested budget in {0.01, 0.05, 0.1}). The recall drop compared to clean inputs shall not exceed the defined safety threshold for safety-critical classes.	The FGSM experiments showed that adversarial perturbations can reduce recall. Since low recall means missed pedestrians, traffic lights, or vehicles, the severity is high. Therefore, adversarial robustness must be explicitly tested and bounded.

→ Pedestrian false negatives are safety-critical, this drop should be limited by a strict robustness requirement.

• a system-level constraint on the response when a low-confidence or anomalous prediction is detected.

→

Constraint ID	Type	Safety constraint	Justification
SC-16	System-level	If the system detects low-confidence, anomalous, inconsistent, or adversarially suspicious perception output, the planner shall not continue normal driving. It shall reduce speed, increase safety distance, request takeover, or perform a minimum-risk maneuver.	Robustness at model level is not enough. If perception becomes unreliable, the system must switch to a safe fallback instead of trusting the classifier output.

4. Residual risk. Even with adversarial training that meets your model-level constraint, what risk remains? Does robust training fully address the UCA and hazard?

→
Even if adversarial training meets the model-level constraint, risk remains.

Residual risk	Explanation
Attack may be stronger than tested budget	The model may be robust for (epsilon = 0.01) or (epsilon = 0.05), but fail for larger perturbations or different norms.
Defense may not generalize	Training against FGSM may not fully protect against stronger attacks such as PGD or physical attacks.
Adaptive attacker risk	An attacker may know the defense and design a perturbation that bypasses it.
Clean accuracy trade-off	Adversarial training can improve robustness but may reduce clean performance.
Detection/response failure	Even if suspicious input is detected, the system may respond too late or incorrectly.
Other perception failures remain	The model can still fail on clean or OOD inputs, not only adversarial ones.

→ Even with adversarial training, residual risk remains. The model may only be robust for the tested ϵ-budget and attack type, while stronger or adaptive attacks may still succeed.

Adversarial training may also reduce clean accuracy and does not address all OOD or sensor-failure cases. Therefore, robust training reduces the risk but does not fully eliminate the UCA or hazard. It must be combined with runtime monitoring and safe system-level fallback behavior.